Statistical Inference and Modeling

Comprehensive Lecture Notes (Lectures 1-9)

Part 1: Foundations and Supervised Learning

Compiled for Comprehensive Understanding

February 12, 2026

Important Note: This document represents the first half of the complete course material covering Lectures 1-9. The second half (Lectures 10-17) will cover advanced topics including Support Vector Machines, Tree-Based Methods, Ensemble Methods, and Unsupervised Learning. This comprehensive lecture note is designed as a textbook-style reference for deep understanding and exam preparation.

Contents

1 Fundamentals of Statistical Learning

1.1 Overview of Machine Learning Paradigms

Machine learning is a field of artificial intelligence concerned with the development of algorithms and statistical models that enable computers to improve their performance on tasks through experience, without being explicitly programmed. The fundamental goal is to learn a function or model from data that can make accurate predictions or decisions on new, unseen data.

1.1.1 Types of Learning Paradigms

1. **Supervised Learning:**

   In supervised learning, we are given a labeled dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ where each input x_i is paired with a corresponding target output y_i. The goal is to learn a function $f : \mathcal{X} \to \mathcal{Y}$ that maps inputs to outputs and generalizes well to unseen data.

   - **Regression:** The target variable y is continuous. Examples include predicting house prices, stock returns, or temperature. The model learns to approximate the relationship between features and a continuous outcome.
   - **Classification:** The target variable y is discrete (categorical). Examples include spam detection, disease diagnosis, or image recognition. The model learns decision boundaries that separate different classes.
   - **Mathematical Formulation:** We seek to minimize a loss function:

     $$\hat{f} = \arg\min_{f \in \mathcal{H}} \sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda R(f)$$

     where L is the loss function, $\mathcal{H}$ is the hypothesis space, and $\lambda R(f)$ is a regularization term to prevent overfitting.

2. **Unsupervised Learning:**

   In unsupervised learning, we have only input data $D = \{x_1, x_2, \ldots, x_n\}$ without corresponding target labels. The goal is to discover the underlying structure, patterns, or distributions in the data.

   - **Clustering:** Group similar data points together. Examples include customer segmentation, document clustering, or gene grouping in bioinformatics.
   - **Dimensionality Reduction:** Reduce the number of features while preserving important information. This helps with visualization, noise reduction, and computational efficiency.
   - **Density Estimation:** Learn the probability distribution of the data, useful for anomaly detection and generative modeling.
   - **Mathematical Formulation:** We typically maximize the likelihood:

     $$\hat{\theta} = \arg\max_{\theta} P(X|\theta)$$

3. **Semi-Supervised Learning:**

   Semi-supervised learning leverages both labeled and unlabeled data. This is particularly useful when labeling is expensive or time-consuming. The key assumption is that the unlabeled data contains structural information that can improve learning.

   - **Assumptions:** The continuity assumption suggests that points close to each other in feature space are likely to have the same label. The manifold assumption suggests that data lies on a lower-dimensional manifold within the high-dimensional space.

4. **Reinforcement Learning (RL):**

   In reinforcement learning, an agent learns to make sequential decisions by interacting with an environment. The agent receives rewards or penalties for its actions and learns to maximize cumulative reward over time.

   - **Components:** State s (current situation), Action a (decision), Reward r (feedback), Policy $\pi(a|s)$ (decision rule).
   - **Applications:** Game playing, robotics, autonomous driving, resource allocation.

1.2 Discriminative vs. Generative Models

Understanding the distinction between discriminative and generative models is crucial for selecting appropriate algorithms for different problems.

Concept: Discriminative Models

Discriminative models directly model the conditional probability $P(Y|X)$. They focus on learning the decision boundary that separates different classes. These models are typically more efficient for classification tasks because they only need to learn the boundary, not the full distribution of each class.
Examples: Logistic Regression, Support Vector Machines (SVM), Neural Networks, K-Nearest Neighbors (KNN).
Advantages:

- Often more accurate for classification when sufficient labeled data is available
- Computationally efficient for prediction
- Flexible in modeling complex decision boundaries

Disadvantages:

- Cannot generate new data points
- Requires labeled data for training
- May not work well with missing data

Analogy: Like learning to distinguish French from Spanish by listening to the sounds and patterns, without necessarily understanding the grammar.

Concept: Generative Models

Generative models model the joint probability $P(X, Y) = P(X|Y)P(Y)$. They learn how data is generated for each class, understanding the underlying distribution. This allows them to generate new data points and handle missing data more gracefully.
Examples: Naive Bayes, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), Gaussian Mixture Models (GMM).
Advantages:

- Can generate new synthetic data
- Can handle missing data better
- Provides probabilistic interpretation
- Can be used for semi-supervised learning

Disadvantages:

- Often less accurate than discriminative models for classification
- Computationally more expensive
- Requires modeling full data distribution

Inference: Use Bayes rule to get the conditional probability:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)} \propto P(X|Y)P(Y)$$

Analogy: Like learning to speak both French and Spanish fluently to distinguish them, understanding the underlying structure of each language.

2 Linear Regression: The Foundation of Predictive Modeling

2.1 Mathematical Formulation and Assumptions

Linear regression is one of the most fundamental and widely-used statistical methods. Despite its simplicity, understanding linear regression deeply provides insights applicable to more complex models.

2.1.1 The Linear Regression Model

The linear regression model assumes a linear relationship between the input features and the continuous target variable:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where:

- $\mathbf{y} \in \mathbb{R}^{n\times 1}$ is the target vector containing n observations
- $\mathbf{X} \in \mathbb{R}^{n\times(p+1)}$ is the design matrix with n observations and $p+1$ columns (including intercept)
- $\boldsymbol{\beta} \in \mathbb{R}^{(p+1)\times 1}$ is the coefficient vector we want to estimate
- $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2\mathbf{I})$ is the error term, assumed to be normally distributed with mean 0 and constant variance σ^2

2.1.2 Key Assumptions

For ordinary least squares (OLS) regression to be valid and for inference to be correct, several assumptions must hold:

1. **Linearity:** The relationship between X and y is linear. Violations can be addressed through feature engineering or non-linear models.
2. **Independence:** Observations are independent. This is violated in time series or clustered data, requiring specialized methods like time series models or mixed-effects models.
3. **Homoscedasticity:** The variance of errors is constant across all levels of X. Violations suggest the need for weighted least squares or robust regression.
4. **Normality:** Errors are normally distributed. This is important for inference (confidence intervals, hypothesis tests) but less critical for point estimation if n is large (Central Limit Theorem).
5. **No Multicollinearity:** Features are not perfectly linearly dependent. High multicollinearity leads to unstable coefficient estimates and inflated standard errors.

2.2 Derivation of the Ordinary Least Squares (OLS) Estimator

The OLS estimator minimizes the sum of squared residuals:

$$L(\boldsymbol{\beta}) = ||\mathbf{y} - \mathbf{X}\boldsymbol{\beta}||_2^2 = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

Derivation: Ordinary Least Squares (OLS)

Expanding the loss function:

$$\begin{aligned} L(\boldsymbol{\beta}) &= (\mathbf{y}^T - \boldsymbol{\beta}^T\mathbf{X}^T)(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \\ &= \mathbf{y}^T\mathbf{y} - \mathbf{y}^T\mathbf{X}\boldsymbol{\beta} - \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{y} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} \end{aligned}$$

Since $\mathbf{y}^T\mathbf{X}\boldsymbol{\beta}$ is a scalar, $(\mathbf{y}^T\mathbf{X}\boldsymbol{\beta})^T = \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{y}$, so:

$$L(\boldsymbol{\beta}) = \mathbf{y}^T\mathbf{y} - 2\boldsymbol{\beta}^T\mathbf{X}^T\mathbf{y} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta}$$

Taking the derivative with respect to $\boldsymbol{\beta}$ and setting it to zero (using matrix calculus rules):

$$\begin{aligned} \nabla_{\boldsymbol{\beta}} L &= -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = \mathbf{0} \\ \mathbf{X}^T\mathbf{X}\boldsymbol{\beta} &= \mathbf{X}^T\mathbf{y} \\ \hat{\boldsymbol{\beta}} &= (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \end{aligned}$$

This is the **Normal Equation**. The matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ is called the **Hat Matrix** or **Projection Matrix**.

2.3 Geometric Interpretation

The OLS solution has an elegant geometric interpretation. The predicted values $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$ are the orthogonal projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$.

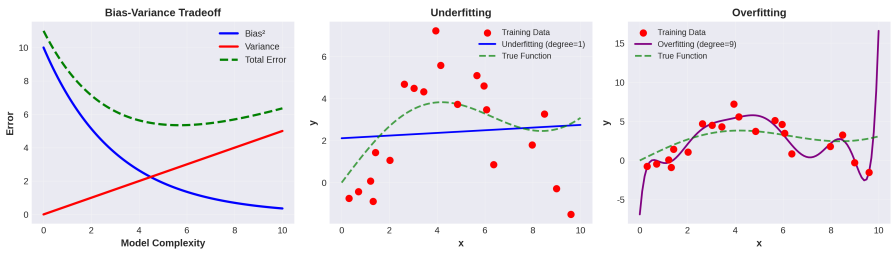

Figure 1: Geometric Interpretation: The predicted vector $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$. The residual vector $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to this space.

This means:

- The residuals $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ are orthogonal to the column space of $\mathbf{X}$
- The residuals satisfy $\mathbf{X}^T\mathbf{e} = \mathbf{0}$
- The fit is the best possible in the least squares sense

2.4 Diagnostics and Influence Analysis

After fitting a linear regression model, it is crucial to assess whether the model assumptions are satisfied and to identify influential observations.

2.4.1 Hat Matrix and Leverage

The hat matrix $\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ has several important properties:

- **Symmetric:** $\mathbf{H}^T = \mathbf{H}$
- **Idempotent:** $\mathbf{H}^2 = \mathbf{H}$
- **Diagonal elements:** $h_{ii} = \mathbf{H}_{ii}$ represent the leverage of observation i
- **Range:** $0 \leq h_{ii} \leq 1$ and $\sum_{i=1}^{n} h_{ii} = p + 1$

High leverage points are observations whose feature values are far from the center of the feature space. These points have the potential to influence the fitted model substantially.

2.4.2 Residuals and Standardized Residuals

The residuals are defined as:

$$e_i = y_i - \hat{y}_i$$

However, residuals have different variances. The standardized residuals are:

$$r_i = \frac{e_i}{\sqrt{\text{MSE}(1 - h_{ii})}}$$

where $\text{MSE} = \frac{\sum e_i^2}{n-p-1}$ is the mean squared error. Standardized residuals should approximately follow a standard normal distribution if the model assumptions are satisfied.

2.4.3 Cook's Distance

Cook's distance measures the influence of the i-th observation on all fitted values. It combines information about both the residual and the leverage:

$$D_i = \frac{\sum_{j=1}^{n}(\hat{y}_j - \hat{y}_{j(i)})^2}{(p+1) \cdot \text{MSE}} = \frac{e_i^2}{(p+1) \cdot \text{MSE}} \left[\frac{h_{ii}}{(1 - h_{ii})^2}\right]$$

where $\hat{y}_{j(i)}$ is the fitted value for observation j when observation i is excluded from the fitting. Generally, observations with $D_i > 1$ are considered influential and warrant further investigation.

3 Model Selection and Validation

3.1 The Bias-Variance Tradeoff

One of the most fundamental concepts in statistical learning is the bias-variance tradeoff. When we fit a model to data, the expected prediction error can be decomposed into three components:

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \text{Bias}^2(\hat{f}) + \text{Var}(\hat{f}) + \sigma^2$$

where:

- **Bias:** $\text{Bias}(\hat{f}) = \mathbb{E}[\hat{f}(X)] - f(X)$ measures how far the expected prediction is from the true function. High bias indicates underfitting.
- **Variance:** $\text{Var}(\hat{f}) = \mathbb{E}[(\hat{f}(X) - \mathbb{E}[\hat{f}(X)])^2]$ measures how much the predictions vary with different training sets. High variance indicates overfitting.
- **Irreducible Error:** σ^2 is the noise inherent in the problem and cannot be reduced by the model.

As we increase model complexity (e.g., adding more features or using more flexible models):

- **Bias decreases:** The model can fit more complex patterns
- **Variance increases:** The model becomes more sensitive to training data fluctuations
- **Optimal complexity:** There is a sweet spot that minimizes total error

3.2 Overfitting and Underfitting

Underfitting occurs when the model is too simple to capture the underlying patterns in the data. This results in high bias and low variance. The model performs poorly on both training and test data.

Overfitting occurs when the model is too complex and fits the noise in the training data. This results in low bias and high variance. The model performs well on training data but poorly on test data.

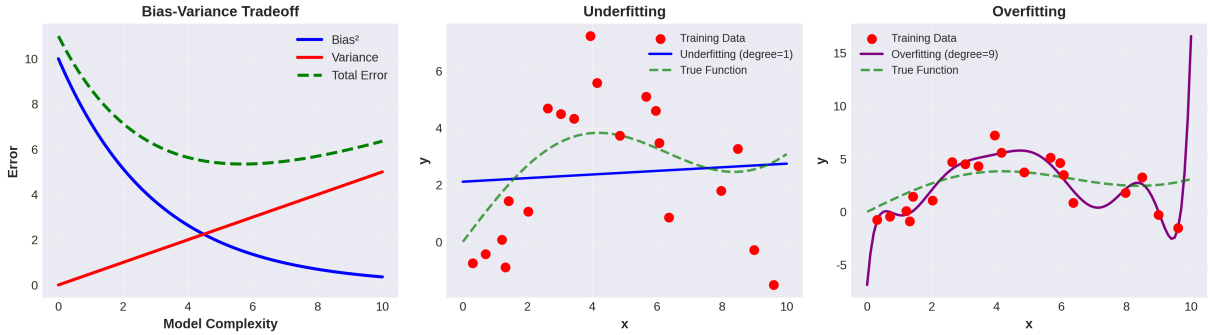

Figure 2: Illustration of bias-variance tradeoff and overfitting/underfitting. Left: As model complexity increases, bias decreases but variance increases. Middle: Underfitting with a linear model. Right: Overfitting with a high-degree polynomial.

3.3 Information Criteria

Information criteria provide a principled way to balance model fit and complexity. They penalize the likelihood by the number of parameters to prevent overfitting.

3.3.1 Akaike Information Criterion (AIC)

AIC is defined as:

$$\text{AIC} = -2\ln(\hat{L}) + 2d$$

where $\hat{L}$ is the maximum likelihood and d is the number of parameters. AIC is asymptotically equivalent to leave-one-out cross-validation and is efficient for prediction. Models with lower AIC are preferred.

3.3.2 Bayesian Information Criterion (BIC)

BIC is defined as:

$$\text{BIC} = -2\ln(\hat{L}) + d\ln(n)$$

BIC is consistent for model selection (it asymptotically selects the true model if it is in the candidate set) and penalizes complexity more heavily than AIC when $n > e^2 \approx 7.4$. BIC is often preferred when the goal is model interpretation rather than prediction.

3.3.3 Corrected AIC (AICc)

For small sample sizes, AIC can be biased. The corrected version is:

$$\text{AICc} = \text{AIC} + \frac{2d(d+1)}{n-d-1}$$

AICc is recommended when $n/d < 40$.

3.4 Cross-Validation Techniques

Cross-validation is a non-parametric approach to estimate the generalization error of a model. It is particularly useful when the sample size is small or when we want to avoid assumptions about the error distribution.

3.4.1 Hold-Out Validation

The simplest approach is to split the data into training and test sets (e.g., 70/30 split). The model is trained on the training set and evaluated on the test set.

Advantages:

- Simple and fast
- Single evaluation

Disadvantages:

- High variance in the estimate (depends on which observations end up in test set)
- Wastes data (test set is not used for training)

3.4.2 K-Fold Cross-Validation

In K-fold CV, the data is divided into K roughly equal-sized folds. The model is trained K times, each time using $K-1$ folds for training and 1 fold for testing. The cross-validation error is the average of the K test errors.

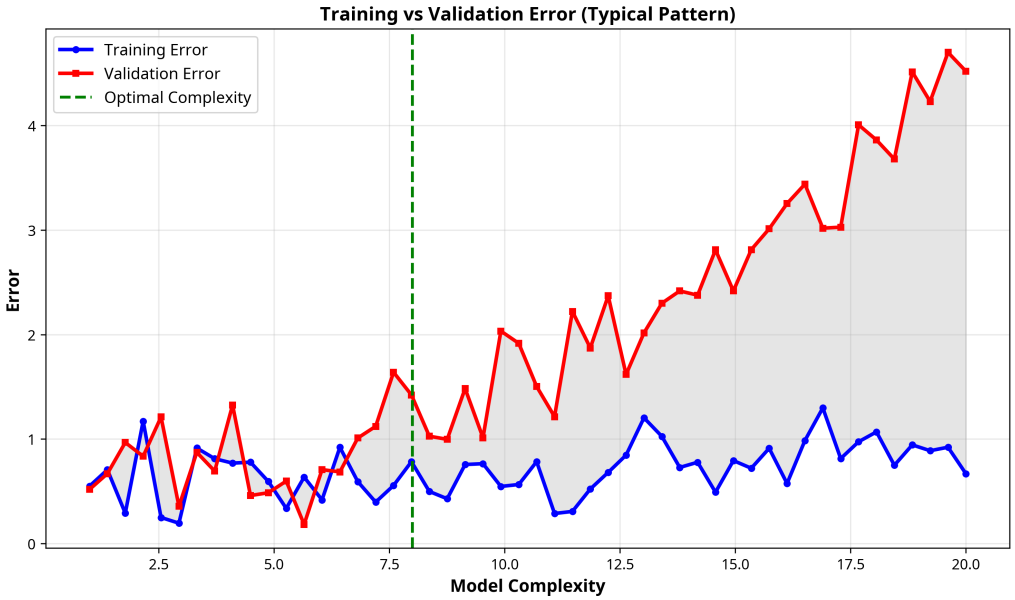

Figure 3: K-Fold Cross-Validation: Data is split into K folds. In each iteration, one fold is used for testing and K-1 folds for training. The final error estimate is the average across all K iterations.

Advantages:

- Lower variance than hold-out validation
- Uses all data for both training and validation
- Computationally feasible for moderate K

Disadvantages:

- Computationally more expensive than hold-out
- Estimates are not independent (folds overlap)

Typical choices are $K = 5$ or $K = 10$. Higher K gives lower bias but higher variance in the CV estimate.

3.4.3 Leave-One-Out Cross-Validation (LOOCV)

LOOCV is the extreme case where $K = n$. Each observation is used as a test set exactly once. While LOOCV has zero bias with respect to the training set size, it has high variance because the n estimates are highly correlated.

For linear regression, LOOCV can be computed efficiently using:

$$\text{CV} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{e_i}{1 - h_{ii}} \right)^2$$

without actually fitting the model n times.

3.5 Regularization: Ridge and Lasso Regression

When multicollinearity is present or when we want to reduce model complexity, regularization techniques add a penalty term to the loss function.

3.5.1 Ridge Regression

Ridge regression adds an L2 penalty on the coefficients:

$$L(\boldsymbol{\beta}) = ||\mathbf{y} - \mathbf{X}\boldsymbol{\beta}||_2^2 + \lambda ||\boldsymbol{\beta}||_2^2$$

The solution is:

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda \mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

Ridge regression:

- Reduces coefficient magnitudes
- Handles multicollinearity by making $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ invertible even when $\mathbf{X}^T\mathbf{X}$ is singular
- Does not perform feature selection (all features remain in the model)
- Requires tuning the regularization parameter λ

3.5.2 Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator) adds an L1 penalty:

$$L(\boldsymbol{\beta}) = ||\mathbf{y} - \mathbf{X}\boldsymbol{\beta}||_2^2 + \lambda ||\boldsymbol{\beta}||_1$$

where $||\boldsymbol{\beta}||_1 = \sum_{j=1}^{p} |\beta_j|$.
Lasso:

- Shrinks some coefficients exactly to zero (feature selection)
- Produces sparse models (fewer non-zero coefficients)
- Useful for interpretability and computational efficiency
- More computationally expensive than ridge (no closed-form solution)

3.5.3 Elastic Net

Elastic Net combines L1 and L2 penalties:

$$L(\boldsymbol{\beta}) = ||\mathbf{y} - \mathbf{X}\boldsymbol{\beta}||_2^2 + \lambda_1 ||\boldsymbol{\beta}||_1 + \lambda_2 ||\boldsymbol{\beta}||_2^2$$

This combines the benefits of both ridge and lasso.

4 Robust Regression

4.1 Motivation and Limitations of OLS

Ordinary least squares regression is optimal under the assumption that errors are normally distributed with constant variance. However, in practice:

- Data often contains outliers (extreme observations)
- Error distributions may have heavy tails
- Variance may not be constant (heteroscedasticity)

Outliers can have a disproportionate influence on OLS estimates because the loss function squares the residuals, giving large weight to large errors. Robust regression methods downweight or ignore outliers.

4.2 M-Estimation

M-estimation generalizes maximum likelihood estimation. Instead of minimizing $\sum(y_i - \hat{y}_i)^2$, we minimize:

$$\sum \rho(e_i)$$

where ρ is a robust loss function. Different choices of ρ give different robust regression methods:

- **Huber Loss:** Quadratic for small residuals, linear for large residuals. Combines the robustness of absolute deviation with the efficiency of squared loss.
- **Tukey Bisquare:** Completely downweights observations with residuals beyond a threshold.
- **Absolute Deviation:** $\rho(e) = |e|$. Minimizes the sum of absolute residuals (L1 regression).

4.3 Iteratively Reweighted Least Squares (IRLS)

A practical algorithm for M-estimation is IRLS:

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

1: Initialize weights $w_i = 1$ for all i
2: **repeat**
3: Fit weighted least squares: $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{W}\mathbf{y}$
4: Compute residuals: $e_i = y_i - \hat{y}_i$
5: Update weights: $w_i = \psi'(e_i/\sigma)/(e_i/\sigma)$ where ψ' is the derivative of the robust loss
6: **until** convergence

5 Classification: Fundamentals and Evaluation

5.1 Classification Problem Setup

In classification, the target variable y takes discrete values (classes). We observe a training set $\{(x_i, y_i)\}_{i=1}^n$ where $y_i \in \{1, 2, \ldots, K\}$ for K classes.

The goal is to learn a classifier $\hat{f}$ that minimizes the misclassification rate on new data:

$$\text{Error Rate} = P(\hat{f}(X) \neq Y)$$

5.2 Confusion Matrix and Classification Metrics

For binary classification, the confusion matrix summarizes predictions vs. actual labels:

		Predicted	
		Negative	Positive
Actual	Negative	TN	FP
	Positive	FN	TP

Table 1: Confusion Matrix for Binary Classification

Important metrics derived from the confusion matrix:

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$ - Overall correctness
- **Sensitivity (Recall):** $\frac{TP}{TP+FN}$ - Proportion of actual positives correctly identified
- **Specificity:** $\frac{TN}{TN+FP}$ - Proportion of actual negatives correctly identified
- **Precision:** $\frac{TP}{TP+FP}$ - Proportion of predicted positives that are correct
- **F1-Score:** $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ - Harmonic mean of precision and recall

5.2.1 Receiver Operating Characteristic (ROC) Curve

The ROC curve plots the True Positive Rate (Sensitivity) against the False Positive Rate (1 - Specificity) as we vary the classification threshold. The Area Under the Curve (AUC) summarizes classifier performance:

- AUC = 1: Perfect classifier
- AUC = 0.5: Random classifier
- AUC = 0: Completely wrong classifier

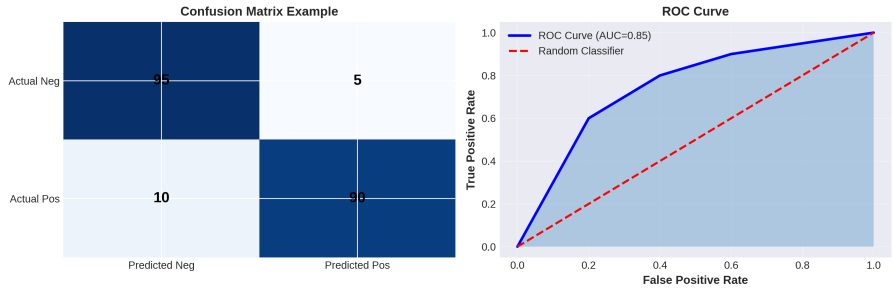

Figure 4: Confusion Matrix and ROC Curve. The ROC curve shows the tradeoff between true positive rate and false positive rate. AUC measures the overall classifier performance.

6 Logistic Regression

6.1 Model Formulation

Logistic regression is a fundamental classification method that models the probability of the positive class using the logistic (sigmoid) function:

$$P(Y = 1|X) = \sigma(\boldsymbol{\beta}^T X) = \frac{1}{1 + e^{-\boldsymbol{\beta}^T X}}$$

The sigmoid function maps any real number to the interval $(0, 1)$, making it suitable for probability modeling. The decision boundary is typically set at probability 0.5, but this can be adjusted based on the cost of different types of errors.

6.2 Geometric Interpretation

The logistic function has several important properties:

- **S-shaped curve:** Smooth transition from 0 to 1
- **Symmetry:** $\sigma(z) + \sigma(-z) = 1$
- **Derivative:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- **Odds:** $\frac{P(Y=1|X)}{P(Y=0|X)} = e^{\boldsymbol{\beta}^T X}$
- **Log-odds:** $\log\left(\frac{P(Y=1|X)}{P(Y=0|X)}\right) = \boldsymbol{\beta}^T X$ (linear in features)

6.3 Maximum Likelihood Estimation

For a binary classification problem with n i.i.d. samples, the likelihood function is:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}$$

where $p_i = \sigma(\boldsymbol{\beta}^T x_i)$.
The log-likelihood is:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

This is also known as the cross-entropy loss or binary cross-entropy.

Derivation: Gradient of Log-Likelihood

Taking the derivative with respect to $\boldsymbol{\beta}$:

$$\begin{aligned}
\frac{\partial \ell}{\partial \boldsymbol{\beta}} &= \sum_{i=1}^{n} \left[y_i \frac{1}{p_i} \frac{\partial p_i}{\partial \boldsymbol{\beta}} - (1 - y_i) \frac{1}{1 - p_i} \frac{\partial p_i}{\partial \boldsymbol{\beta}} \right] \\
&= \sum_{i=1}^{n} \left[y_i \frac{1}{p_i} - (1 - y_i) \frac{1}{1 - p_i} \right] p_i(1 - p_i) x_i \\
&= \sum_{i=1}^{n} [y_i(1 - p_i) - (1 - y_i)p_i] x_i \\
&= \sum_{i=1}^{n} (y_i - p_i) x_i
\end{aligned}$$

In matrix form: $\nabla_{\boldsymbol{\beta}} \ell = \mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}})$

6.4 Optimization: Gradient Descent

Unlike linear regression, logistic regression has no closed-form solution. We use iterative optimization algorithms such as gradient descent:

Convergence: The log-likelihood is concave, so gradient descent is guaranteed to find the global optimum. In practice, we check convergence by monitoring the change in log-likelihood or coefficients.

Algorithm 2 Gradient Descent for Logistic Regression

1: Initialize $\boldsymbol{\beta} \leftarrow \mathbf{0}$
2: Set learning rate α (e.g., 0.01)
3: **repeat**
4: Compute predictions: $\hat{\mathbf{y}} = \sigma(\mathbf{X}\boldsymbol{\beta})$
5: Compute gradient: $\nabla = \mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}})$
6: Update: $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} + \alpha\nabla$ (gradient ascent for maximizing likelihood)
7: Check convergence
8: **until** convergence or max iterations

6.5 Multi-class Logistic Regression

For $K > 2$ classes, we use the softmax function:

$$P(Y = k|X) = \frac{e^{\boldsymbol{\beta}_k^T X}}{\sum_{j=1}^{K} e^{\boldsymbol{\beta}_j^T X}}$$

This is also called multinomial logistic regression. The loss function is the multi-class cross-entropy:

$$\ell = -\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik} \log(P(Y = k|x_i))$$

where $y_{ik} = 1$ if observation i belongs to class k, and 0 otherwise (one-hot encoding).

7 Generative Classifiers: LDA and QDA

7.1 Linear Discriminant Analysis (LDA)

LDA is a generative classifier that assumes each class follows a multivariate normal distribution with the same covariance matrix across classes.

7.1.1 Model Assumptions

For class k:

$$X|Y = k \sim \mathcal{N}(\mu_k, \Sigma)$$

where μ_k is the class-specific mean and Σ is the shared covariance matrix. The prior probability of class k is $\pi_k = P(Y = k)$.

7.1.2 Derivation of LDA Discriminant Function

Using Bayes rule:

$$P(Y = k|X) \propto P(X|Y = k)P(Y = k)$$

Taking the log and removing terms that don't depend on k:

Derivation: LDA Discriminant Function

$$\begin{aligned}
\delta_k(x) &= \log P(X|Y = k) + \log P(Y = k) \\
&= -\frac{1}{2}(x - \mu_k)^T\Sigma^{-1}(x - \mu_k) + \log \pi_k + \text{const} \\
&= -\frac{1}{2}x^T\Sigma^{-1}x + x^T\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log \pi_k + \text{const}
\end{aligned}$$

The first term $-\frac{1}{2}x^T\Sigma^{-1}x$ is the same for all classes, so we can drop it:

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

This is **linear** in x.

7.2 Quadratic Discriminant Analysis (QDA)

QDA relaxes the assumption of equal covariances. Each class has its own covariance matrix:

$$X|Y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$$

The discriminant function becomes:

$$\delta_k(x) = -\frac{1}{2} x^T \Sigma_k^{-1} x + x^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \log |\Sigma_k| + \log \pi_k$$

The first term $-\frac{1}{2} x^T \Sigma_k^{-1} x$ now depends on k, making the decision boundary **quadratic**.

7.2.1 Comparison of LDA and QDA

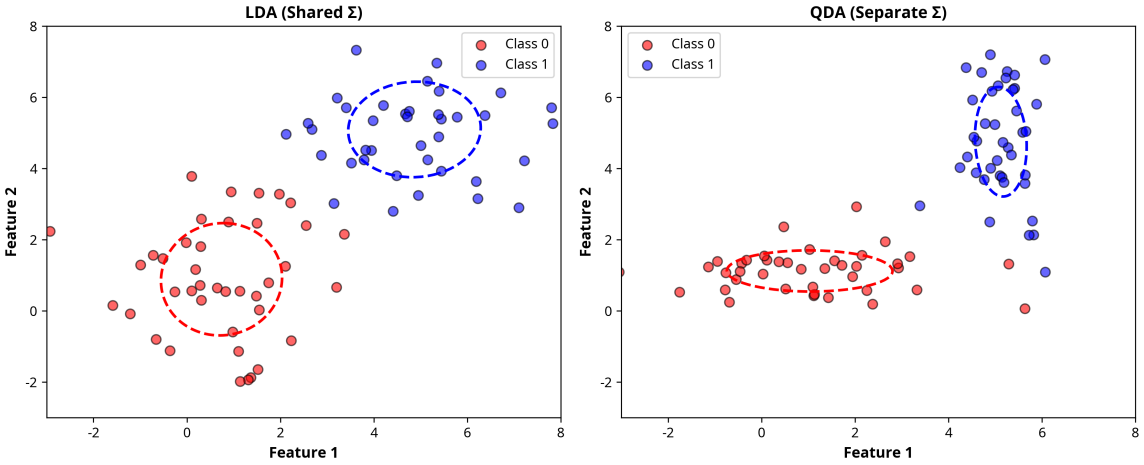

Figure 5: LDA vs QDA Decision Boundaries. LDA assumes shared covariance, resulting in linear boundaries. QDA allows class-specific covariances, resulting in quadratic boundaries that can be more flexible.

Aspect	**LDA**	**QDA**
Covariance	Shared across classes	Class-specific
Decision Boundary	Linear	Quadratic
Parameters	$O(p)$ per class	$O(p^2)$ per class
Bias	Higher	Lower
Variance	Lower	Higher
Sample Size	Works with smaller n	Needs larger n

Table 2: Comparison of LDA and QDA

Naive Bayes

Naive Bayes is a generative classifier that makes a strong independence assumption: given the class label, the features are conditionally independent.

7.2.2 Conditional Independence Assumption

$$P(X_1, X_2, \ldots, X_p|Y = k) = \prod_{j=1}^{p} P(X_j|Y = k)$$

This dramatically simplifies computation and reduces the number of parameters from $O(2^p)$ to $O(p)$.

7.2.3 Classification Rule

$$P(Y = k|X) \propto P(Y = k) \prod_{j=1}^{p} P(X_j|Y = k)$$

For continuous features, we typically assume $X_j|Y = k \sim \mathcal{N}(\mu_{jk}, \sigma_{jk}^2)$.

7.2.4 Advantages and Disadvantages

Advantages:

- Simple and fast
- Works well with high-dimensional data
- Requires relatively small training set
- Probabilistic interpretation

Disadvantages:

- Strong independence assumption often violated in practice
- Can underestimate probabilities (especially with small sample sizes)
- May not be as accurate as more sophisticated methods

8 Neural Networks: Introduction

8.1 Motivation and Biological Inspiration

Neural networks are inspired by the structure and function of biological neurons. A single artificial neuron computes a weighted sum of inputs and applies a non-linear activation function:

$$a = g(z) = g(\mathbf{w}^T\mathbf{x} + b)$$

where $\mathbf{w}$ are weights, b is the bias, and g is the activation function. Multiple neurons arranged in layers form a neural network.

8.2 Network Architecture

A feedforward neural network consists of:

- **Input Layer:** Receives the feature vector $\mathbf{x} \in \mathbb{R}^p$
- **Hidden Layers:** Intermediate layers that learn representations. Each layer applies a linear transformation followed by a non-linear activation.
- **Output Layer:** Produces predictions. For regression, typically linear activation; for classification, sigmoid or softmax.

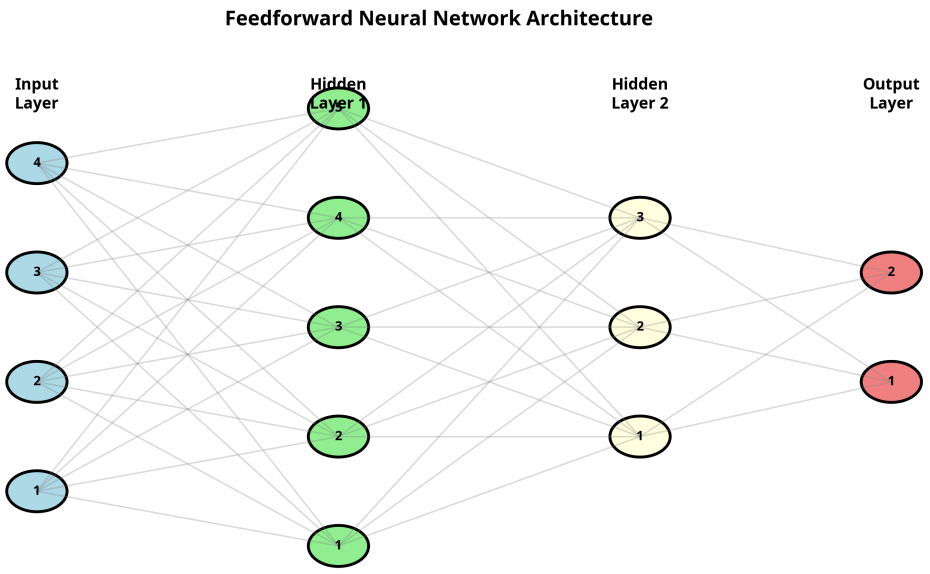

Figure 6: Feedforward Neural Network Architecture. Information flows from input layer through hidden layers to the output layer. Each connection has an associated weight.

8.3 Activation Functions

Activation functions introduce non-linearity, allowing networks to learn complex patterns.

8.3.1 Common Activation Functions

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$. Maps to $(0, 1)$. Used in output layer for binary classification. Suffers from vanishing gradient problem.
- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Maps to $(-1, 1)$. Zero-centered, often better than sigmoid for hidden layers.
- **ReLU (Rectified Linear Unit):** $\text{ReLU}(z) = \max(0, z)$. Simple and efficient. Avoids vanishing gradient. Most popular for hidden layers.
- **Leaky ReLU:** $\text{LeakyReLU}(z) = \max(\alpha z, z)$ where $\alpha \approx 0.01$. Allows small negative values, avoiding dead neurons.
- **Softmax:** $\text{softmax}(z_k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$. Maps to probability distribution. Used in output layer for multi-class classification.

8.4 Forward Propagation

For a network with L layers, forward propagation computes:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = g^{(l)}(z^{(l)})$$

where $a^{(0)} = \mathbf{x}$ is the input, and $a^{(L)}$ is the output.

8.5 Loss Functions

- **Regression:** Mean Squared Error (MSE) or Mean Absolute Error (MAE)
- **Binary Classification:** Binary Cross-Entropy
- **Multi-class Classification:** Categorical Cross-Entropy

8.6 Backpropagation and Gradient Descent

Backpropagation is an efficient algorithm for computing gradients using the chain rule. It propagates errors backward through the network to compute gradients for all weights and biases.

Algorithm 3 Backpropagation Algorithm

1: Forward pass: Compute $a^{(l)}$ for all layers
2: Compute loss: $L = \text{loss}(a^{(L)}, y)$
3: Backward pass: Compute $\delta^{(L)} = \nabla_{a^{(L)}} L$
4: **for** $l = L - 1$ down to 1 **do**
5: Compute $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot g'(z^{(l)})$
6: Compute gradients: $\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$
7: **end for**
8: Update weights: $W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial L}{\partial W^{(l)}}$

8.7 Challenges in Training Deep Networks

- **Vanishing Gradient:** Gradients become very small in early layers, slowing learning. ReLU and batch normalization help.
- **Exploding Gradient:** Gradients become very large, causing instability. Gradient clipping helps.
- **Overfitting:** Deep networks have many parameters and can easily overfit. Regularization, dropout, and early stopping help.
- **Computational Cost:** Training deep networks requires significant computational resources.

9 K-Nearest Neighbors (KNN)

9.1 Principle and Intuition

K-Nearest Neighbors is a simple, non-parametric method based on a fundamental assumption: similar inputs have similar outputs. For a test point, KNN finds the K nearest training points (neighbors) and makes a prediction based on them.

9.2 Algorithm

Algorithm 4 K-Nearest Neighbors Prediction

Require: Training data $D = \{(x_i, y_i)\}_{i=1}^n$, Test point x^*, Hyperparameter K
1: Calculate distance $d(x^*, x_i)$ for all $x_i \in D$ (typically Euclidean distance)
2: Sort distances and identify the K nearest neighbors $N_K(x^*)$
3: **if** Regression **then**
4: $\hat{y} = \frac{1}{K} \sum_{i \in N_K(x^*)} y_i$ (average of neighbors)
5: **else if** Classification **then**
6: $\hat{y} = \text{Mode}(\{y_i : i \in N_K(x^*)\})$ (majority vote)
7: **end if**

9.3 Distance Metrics

The choice of distance metric is crucial:

- **Euclidean Distance:** $d(x, x') = \sqrt{\sum_{j=1}^{p}(x_j - x'_j)^2}$. Most common, assumes continuous features.
- **Manhattan Distance:** $d(x, x') = \sum_{j=1}^{p} |x_j - x'_j|$. More robust to outliers.
- **Hamming Distance:** For categorical features, counts the number of differing positions.
- **Minkowski Distance:** Generalization: $d(x, x') = \left(\sum_{j=1}^{p} |x_j - x'_j|^r\right)^{1/r}$

9.4 Feature Scaling

Warning: Critical: Feature Scaling is Essential

Since KNN relies on distance, features with large scales will dominate the distance calculation. Always apply feature scaling before using KNN:
Z-Score Normalization: $x'_j = \frac{x_j - \mu_j}{\sigma_j}$
Min-Max Scaling: $x'_j = \frac{x_j - \min_j}{\max_j - \min_j}$
Without scaling, a feature with range $[0, 1000]$ will dominate a feature with range $[0, 1]$, leading to biased predictions.

9.5 Hyperparameter Selection

The choice of K significantly affects performance:

- **Small K (e.g., $K = 1$):** Low bias, high variance. Sensitive to noise and outliers. May overfit.
- **Large K (e.g., $K = n$):** High bias, low variance. May underfit. Predictions become less local.
- **Typical choices:** $K = 3, 5, 10$ or $K = \sqrt{n}$

Cross-validation should be used to select the optimal K.

9.6 Advantages and Disadvantages

Advantages:

- Simple to understand and implement
- No training phase (lazy learner)
- Works for both regression and classification
- Can handle non-linear patterns
- No assumptions about data distribution

Disadvantages:

- Computationally expensive at prediction time (must compute distances to all training points)
- Sensitive to feature scaling
- Sensitive to outliers
- High memory requirements (must store all training data)
- Poor performance in high-dimensional spaces (curse of dimensionality)

9.7 Decision Boundaries

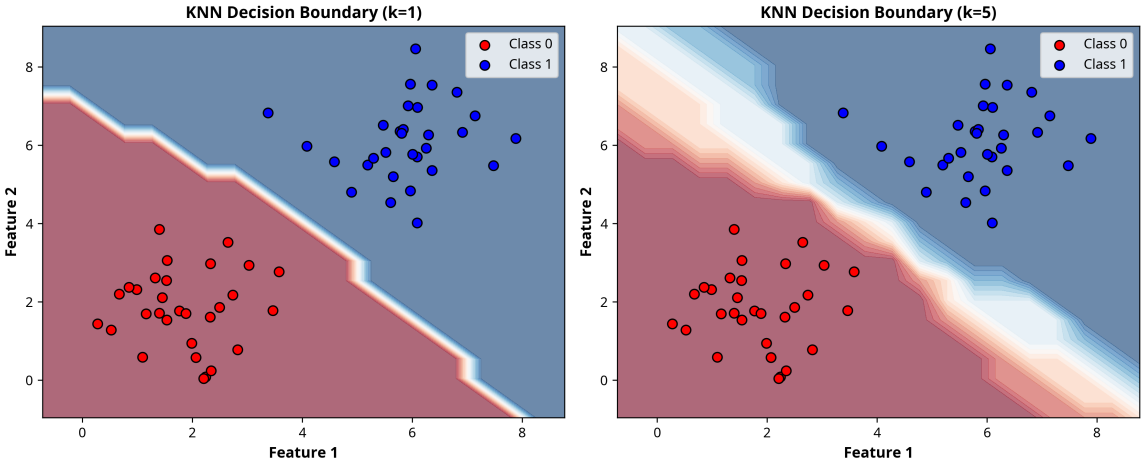

Figure 7: KNN Decision Boundaries for different values of K. With $K = 1$, the boundary is very wiggly and overfits. With $K = 5$, the boundary is smoother and generalizes better.

9.8 Curse of Dimensionality

In high-dimensional spaces, KNN performs poorly because:

- Most points become equidistant from each other
- The concept of "nearest" neighbor becomes less meaningful
- Requires exponentially more data to maintain the same density

Dimensionality reduction techniques (PCA, feature selection) can help mitigate this issue.

10 Summary and Connections

This first part of the course has covered the fundamental concepts and methods in statistical learning:

1. **Foundations:** Understanding learning paradigms, bias-variance tradeoff, and model selection
2. **Regression:** Linear regression as the foundation, with extensions to robust regression
3. **Classification:** Logistic regression, generative classifiers (LDA, QDA, Naive Bayes), and non-parametric methods (KNN)
4. **Neural Networks:** Introduction to flexible function approximators

These methods form the building blocks for more advanced techniques covered in Part 2, including ensemble methods, support vector machines, and unsupervised learning.

10.1 Key Takeaways

- **No Free Lunch:** No single algorithm is best for all problems. Choose based on data characteristics and problem requirements.
- **Bias-Variance Tradeoff:** Understand the tradeoff between model complexity and generalization.
- **Validation is Critical:** Always use cross-validation to estimate generalization error.

- **Feature Engineering:** The quality of features often matters more than the choice of algorithm.
- **Interpretability vs. Accuracy:** Simple models are often preferred for interpretability, while complex models may achieve higher accuracy.